

Data Diversity Matters for Robust Instruction Tuning

Anonymous ACL submission

Abstract

Recent works have shown that by curating high quality and diverse instruction tuning datasets, we can significantly improve instruction-following capabilities. However, creating such datasets is difficult and most works rely on manual curation or proprietary language models. Automatic data curation is difficult as it is still not clear how we can define diversity for instruction tuning, how diversity and quality depend on one other, and how we can optimize dataset quality and diversity. To resolve these issue, we propose a new algorithm, Quality-Diversity Instruction Tuning (QDIT). QDIT provides a simple method to simultaneously control dataset diversity and quality, allowing us to conduct an in-depth study on the effect of diversity and quality on instruction tuning performance. From this study we draw two key insights (1) there is a natural tradeoff between data diversity and quality and (2) increasing data diversity significantly improves the worst case instruction following performance, therefore improving robustness. We validate the performance of QDIT on several large scale instruction tuning datasets, where we find it can substantially improve worst and average case performance compared to quality-driven data selection.

1 Introduction

Large pre-trained language models have demonstrated a remarkable ability to perform a wide range of natural language processing tasks (Vaswani et al., 2017; Devlin et al., 2018; Radford et al., 2019; He et al., 2020; Brown et al., 2020). Although these models are powerful, pre-trained models such as GPT-3 can be quite difficult to work with and often do not follow user instructions (Brown et al., 2020). To unlock instruction-following capabilities, researchers have turned to instruction tuning, in which language models are trained to follow instructions on a small set of example instruction-response pairs (Mishra et al., 2021; Wei et al., 2021;

Sanh et al., 2021; Wang et al., 2022b). Instruction tuning (IFT) has become extremely popular, as it provides a simple way for researchers to train powerful and aligned language models (Taori et al., 2023; Chiang et al., 2023; Xu et al., 2023).

Although initial works apply instruction tuning to large-scale datasets, it has recently been found that a small set of well chosen instruction-response pairs is sufficient for good performance. In particular, Zhou et al. (2023) showed that by training on only 1000 instructions manually selected or crafted by experts, superior performance can be achieved compared to training on larger datasets. Training with a small dataset has the added benefits of lowering training costs and enabling faster iteration. Although such manual data selection is not scalable, this work raises an important question: How can we automatically select an instruction tuning dataset?

Recent work on dataset curation have identified two characteristics that an instruction tuning dataset should have: (1) the instruction responses should be high quality (Chen et al., 2023; Peng et al., 2023) and (2) the instructions should cover a wide range of tasks (i.e. be diverse) (Wei et al., 2021; Zhou et al., 2023; Gudibande et al., 2023). To curate high quality datasets, researchers have used proprietary LLMs to measure the quality of each data point in the dataset and then select only the highest quality data points. To improve dataset diversity, researchers have manually selected instructions that cover a wide range of topics and formats (Zhou et al., 2023; Ivison et al., 2023). While both of these methods enhance instruction-following capabilities, it is not clear how we can select high quality and diverse datasets without relying on manual curation from human experts, a process that is time-consuming and expensive.

In pursuit of this goal, we propose a new algorithm, QDIT, to measure and optimize the diversity and quality of instruction tuning datasets. QDIT

measures diversity using the facility location function (Cornuéjols et al., 1983). The facility location function provides an intuitive measure of subset diversity, as it essentially measures how well represented each data point in the full dataset is by the data points in the selected subset. With this diversity function and quality functions from prior works, we then define a dataset’s quality-diversity score as a simple linear combination of dataset quality and diversity. To optimize the quality-diversity score, QDIT employs a greedy strategy, where the data point that will improve the joint quality-diversity score the most is selected at each time step (Nemhauser et al., 1978). This procedure is extremely efficient, and can easily scale to datasets with millions of instruction.

QDIT provides an effective way to control the diversity and quality of the instruction tuning dataset, allowing us to conduct an in-depth study of diversity and quality in instruction tuning. From this study we identify two key findings: (1) there is an inherent tradeoff between dataset diversity and dataset quality and (2) improving dataset diversity primarily improves the worst and average case instruction following ability, while not affecting best case instruction following ability much. Based on these finding, we are able to use QDIT to optimize the quality-diversity tradeoff, improving worst case performance while maintaining or improving best case and average performance for robust instruction following. We extensively validate our results on five large-scale instruction tuning datasets.

2 Related Work

There have been several works that attempt to improve the quality and diversity of IFT datasets.

◊ **Manual Data Selection.** Several works have shown that superior instruction-following capabilities can be unlocked by carefully selecting and writing instruction-response pairs (Zhou et al., 2023; Touvron et al., 2023; Wang et al., 2023a; Ivison et al., 2023). To select such data quality and diversity are emphasized, with data from various scientific fields and internet forums being selected. It is not clear how such datasets can be automatically selected.

◊ **Distilling Closed Models.** To reduce the human effort required for dataset creation, researchers have used powerful proprietary LLMs such as GPT-4 to create instruction tuning datasets (Taori et al., 2023; Peng et al., 2023; Chia et al., 2023). Again

researchers found dataset quality (Peng et al., 2023) and diversity (Xu et al., 2023; Li et al., 2023b) to be most important. Although resulting in powerful datasets, the reliance on proprietary language models in these works is expensive and may raise legal concerns (Wang et al., 2023b).

◊ **Automatic Data Selection for Instruction Tuning.** Due to the aforementioned issues, in this paper we focus on automatic selection of smaller instruction tuning datasets from larger ones. Chen et al. (2023); Dong et al. (2023b) show that by rating the quality of each data point and training on the highest quality data points, downstream performance can be significantly improved. Li et al. (2023a) propose to select instructions based on difficulty. Wang et al. (2022a); Liu et al. (2023) attempt to increase diversity by restricting the distance between selected points to be larger than a given threshold. We find that this method does not necessarily lead to a significant increase in diversity, and compare QDIT to similar approaches in our experiments.

Our work is also related to several works that seek to increase diversity in NLP, but do not consider data quality (Kumar et al., 2019; Kirchhoff and Bilmes, 2014; Das et al., 2023; Maharana et al., 2023). Concurrently, Bhatt et al. (2024); Wang et al. (2024) analyze the efficacy of various diversity and uncertainty metrics for IFT, but do not consider dataset quality.

3 Methodology

Before presenting QDIT, we discuss how to quantify instruction diversity and instruction-response quality.

3.1 Quantifying Dataset Diversity and Quality

Given a set $A \subseteq V$, a natural way to measure the diversity of the set A with respect to V is by the facility location function (Cornuéjols et al., 1983)

$$d(A) = \sum_{v \in V} \max_{a \in A} \text{sim}(a, v), \quad (1)$$

where $\text{sim}(a, v)$ refers to the similarity of a and v . In QDIT, we use the cosine similarity of instruction embeddings as the similarity function in (1), where the instruction embeddings are computed with sentence transformers (Reimers and Gurevych, 2019). See Appendix A for more details. Intuitively, we can see that a set A that has a high diversity score $d(A)$ will have an $a \in A$ close to each $v \in V$ and will therefore be representative of the set V .

To measure dataset quality, we follow prior works and measure the quality of each (instruction, response) pair using a large language model such as ChatGPT (Chen et al., 2023) or measure the quality of each data point using a scoring model trained on large amounts of human preference data (Ouyang et al., 2022; Bai et al., 2022). We refer to such a function with $q(\cdot)$, and measure a dataset’s overall quality by averaging the quality score of each data point.

3.2 Quality-Diversity Instruction Tuning

In order to simultaneously control quality and diversity of the selected data, we propose a linear combination of quality and diversity as the quality-diversity (Q-D) score:

$$f(a|A, \alpha) = (1 - \alpha)d(a|A) + \alpha q(a),$$

where $\alpha \in [0, 1]$ is a hyperparameter controlling the tradeoff between quality and diversity.

To optimize the Q-D score of the selected data, we consider a greedy algorithm – named QDIT shown in Algorithm 1. Specifically, at each iteration, we select the data point that most increases the Q-D score of the current subset. When $\alpha = 0$, the greedy algorithm achieves the best possible approximation ratio (in the worst case) that a polynomial time algorithm can achieve. See more details in (Nemhauser et al., 1978). We remark that when $\alpha = 1$, QDIT is reduced to the quality driven selection algorithm proposed in Chen et al. (2023) and when $\alpha = 0$, QDIT is reduced the classical greedy algorithm (Nemhauser et al., 1978).

Algorithm 1 QDIT Data Selection: Select a subset of K data points from N data points

Require: K, α

```

1:  $A \leftarrow \emptyset$ 
2:  $R \leftarrow V$ 
3:  $N \leftarrow 0$ 
4: while  $N < K$  do
5:    $a \leftarrow \operatorname{argmax}_{a \in R} f(a|A, \alpha)$ 
6:    $A \leftarrow A \cup \{a\}$ 
7:    $R \leftarrow R \setminus a$ 
8:    $N \leftarrow N + 1$ 
9: end while
```

Once we finish the selection, we apply instruction tuning on the selected data.

Computational Complexity of QDIT. Finding $a \in A$ that maximizes the quality-diversity score

at each time step has complexity $\mathcal{O}(|V|^3)$, leading to a total complexity of $\mathcal{O}(|V|^3 K)$ for QDIT based data selection. This is problematic, since for some datasets $|V|$ is greater than 1 million. We take several approaches to reduce data selection cost: (1) parallelization on GPUs, (2) employing the lazy greedy selection algorithm of Minoux (2005), and (3) sub-sampling according to Mirzasoleiman et al. (2015). These methods can be employed simultaneously, allowing us to select from millions of data points within a few hours. We provide more details in Appendix B.

4 Experiments

We first analyze how the QDIT algorithm affects dataset quality and diversity and then present our main results and analysis.

4.1 Preliminary Analysis

To verify that QDIT can indeed control the quality and diversity of the selected dataset, we first investigate whether the facility location function is aligned with our intuitive understanding of diversity. Next, we study how the α parameter affects the dataset’s quality and diversity. For this analysis, we apply the QDIT algorithm to the Alpaca dataset (Taori et al., 2023).

To qualitatively verify that the facility location function is aligned with dataset diversity, we use the Berkeley Neural Parser (Kitaev and Klein, 2018; Kitaev et al., 2019) to extract the root verb and first direct noun from each instruction in the Alpaca dataset. We then plot the distribution of verb-noun pairs in Figure 1 for random, quality driven, and QDIT data selection. From Figure 1, we observe that selecting based on the facility location function indeed improves the dataset diversity compared to random selection, as more verb-noun pairs (1347 vs 1308) are included in the dataset and the dataset becomes more uniform. On the other hand, selecting based on quality alone decreases dataset diversity.

Next we plot how α in QDIT affects dataset quality and diversity of 3K selected points in Figure 2. Similar figures for other datasets can be found in Appendix C. From these figures, we can observe that there is a tradeoff between quality and diversity, and that QDIT allows us to smoothly control this tradeoff. Moreover, we observe that QDIT is able to improve diversity without significantly decreasing data quality. Altogether, these results

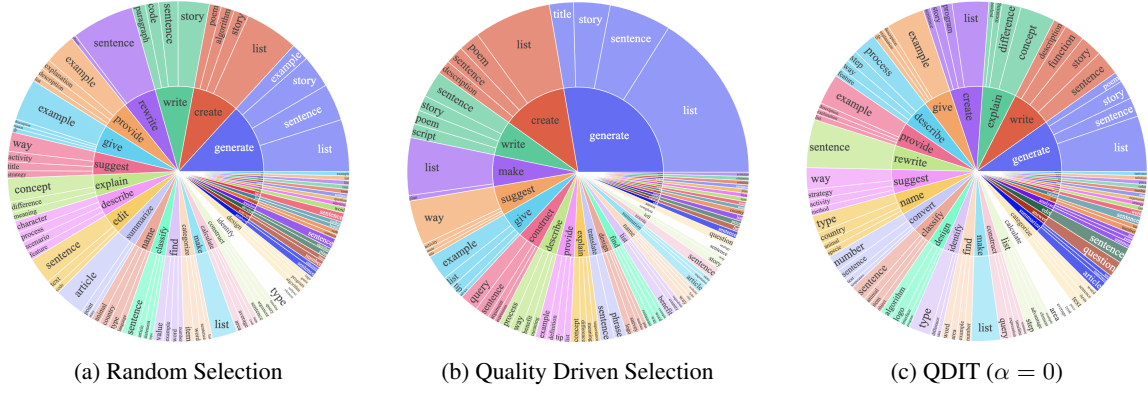


Figure 1: Distribution of root verbs and first nouns selected by different algorithms. The dataset size is 3000.

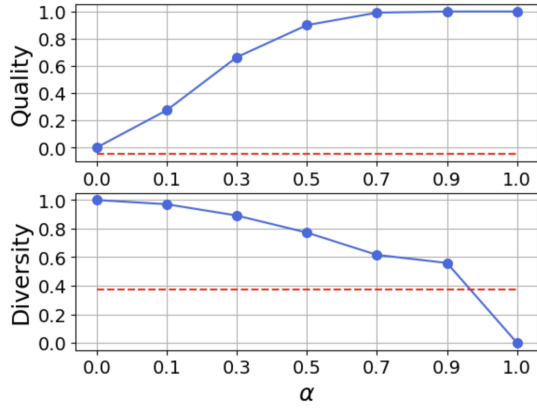


Figure 2: Effect of α on QDIT’s dataset quality and diversity. The red line represents random selection.

indicate that QDIT is a practical way to control dataset diversity and quality.

4.2 Experimental Setup

Now that we can control quality and diversity with QDIT, we seek to study how dataset quality and diversity affect instruction following ability.

✦ **Training Setup.** We use QDIT on two small instruction tuning datasets: Alpaca 52K (Taori et al., 2023), and Dolly 15K (Conover et al., 2023) as well as three large scale datasets: Ultrachat 1.3M (Ding et al., 2023), LMSYS-Chat 1M (Zheng et al., 2023), and a combined dataset of Alpaca 52K, Dolly 15K, and the OIG-small-chip2 dataset (210K). We refer to this dataset as “Mixed 270K”. For each large dataset we select a dataset size of 10K points. For each small dataset size we follow the small data setting from Chen et al. (2023) and select $\sim 5\%$ of the original dataset.

To measure instruction-response quality, we use the provided ChatGPT quality scores from Chen et al. (2023) for Alpaca and for all other datasets we use the reward model from Dong et al. (2023a),

which is trained on the Anthropic Helpful Harmless dataset and achieves a test accuracy of over 75%. For our main experiments we follow the training procedure from Taori et al. (2023) and use LLaMA-1 7B as our base model (Touvron et al., 2023). In all settings we use the same number of training epochs, meaning that the training cost is proportional to the number of training instructions. Complete details can be found in Appendix D.

✦ **Evaluation.** We evaluate the trained models in two main ways: through LLM-based pairwise comparison (Dubois et al., 2023) and by using a reward model. For pairwise comparison, we evaluate the trained model versus a variety of reference models, employing Claude 2 as the judge on five evaluation sets: InstructEval (Wang et al., 2022a), WizardLM (Xu et al., 2023), Vicuna (Chiang et al., 2023), Koala (Geng et al., 2023), and a set of 200 examples manually curated from the ShareGPT dataset. In order to mitigate the effects of the judge LLM’s positional bias, we evaluate the responses in both orders (i.e. QDIT response shown first and QDIT response shown second). We then follow Chen et al. (2023) and measure performance according to winning score ($\frac{\# \text{Win} - \# \text{Lose}}{\text{Total comparisons}} + 1$). Detailed comparison plots can be found in Appendix E. In addition to language model based evaluation, we evaluate our models based on the reward score achieved on each evaluation dataset. We refer to this score as “HH Score.”

✦ **Evaluating Robustness.** Beyond evaluating average performance on the test dataset, we also evaluate the worst and best case performance of each model. We can evaluate the worst case performance with the HH Score by calculating the average score achieved on the worst 10% of instructions for each model (note that the worst instructions can change depending on the model). Similarly, we can evalu-

Table 1: Instruction Tuning results. Random 50K refers to a model trained on a randomly sampled set of 50K points on the corresponding dataset. The top and bottom 10% winning and losing score is versus the Alpaca 52K.

Ultrachat 1.3M	Average Performance			Worst Case Performance		Best Case Performance	
	Winning Score vs. Alpaca 52K $\uparrow$	Winning Score vs. Random 50K $\uparrow$	HH Score Mean $\uparrow$	Lowest 10% HH Score $\uparrow$	Lowest 10% Winning Score $\uparrow$	Top 10% HH Score $\uparrow$	Top 10% Winning Score $\uparrow$
Random 10K	1.138	0.969	6.219	2.620	1.074	9.526	1.167
Quality 10K	1.224	1.013	6.961	3.405	1.175	10.454	1.303
QDIT 10K ($\alpha = 0.7$)	1.226	1.038	6.993	3.497	1.280	10.454	1.293
Mixed 270K	Average Performance			Worst Case Performance		Best Case Performance	
	Winning Score vs. Alpaca 52K $\uparrow$	Winning Score vs. Random 50K $\uparrow$	HH Score Mean $\uparrow$	Lowest 10% HH Score $\uparrow$	Lowest 10% Winning Score $\uparrow$	Top 10% HH Score $\uparrow$	Top 10% Winning Score $\uparrow$
Random 10K	0.899	0.986	5.443	2.260	0.940	8.80	0.896
Quality 10K	0.959	1.04	6.140	2.973	0.989	9.670	0.984
QDIT 10K ($\alpha = 0.9$)	0.987	1.083	6.276	3.054	0.973	9.676	1.044
LMSYS 1M	Average Performance			Worst Case Performance		Best Case Performance	
	Winning Score vs. Alpaca 52K $\uparrow$	Winning Score vs. Random 50K $\uparrow$	HH Score Mean $\uparrow$	Lowest 10% HH Score $\uparrow$	Lowest 10% Winning Score $\uparrow$	Top 10% HH Score $\uparrow$	Top 10% Winning Score $\uparrow$
Random 10K	1.113	1.0	6.176	2.575	1.057	9.589	1.187
Quality 10K	1.198	1.137	7.066	3.391	1.052	10.39	1.284
QDIT 10K ($\alpha = 0.7$)	1.224	1.149	7.0	3.551	1.149	10.34	1.343

Table 2: Instruction Tuning results on small datasets. The setting is the same as Table 1.

Alpaca 52K	Average Performance			Worst Case Performance		Best Case Performance	
	Winning Score vs. Alpaca 52K $\uparrow$	Winning Score vs. Random 50K $\uparrow$	HH Score Mean $\uparrow$	Lowest 10% HH Score $\uparrow$	Lowest 10% Winning Score $\uparrow$	Top 10% HH Score $\uparrow$	Top 10% Winning Score $\uparrow$
Random 3K	0.929	-	5.486	2.105	0.931	8.986	0.937
Quality 3K	0.920	-	5.629	2.306	0.873	9.073	0.934
QDIT 3K ($\alpha = 0.7$)	1.026	-	5.661	2.513	0.924	8.791	1.056
Dolly 15K	Average Performance			Worst Case Performance		Best Case Performance	
	Winning Score vs. Alpaca 52K $\uparrow$	Winning Score vs. Random 15K $\uparrow$	HH Score Mean $\uparrow$	Lowest 10% HH Score $\uparrow$	Lowest 10% Winning Score $\uparrow$	Top 10% HH Score $\uparrow$	Top 10% Winning Score $\uparrow$
Random 1K	0.64	0.72	4.632	1.474	0.632	6.144	0.645
Quality 1K	0.71	0.83	5.389	2.082	0.681	7.751	0.746
QDIT 1K ($\alpha = 0.7$)	0.739	0.874	5.495	2.229	0.742	7.873	0.872

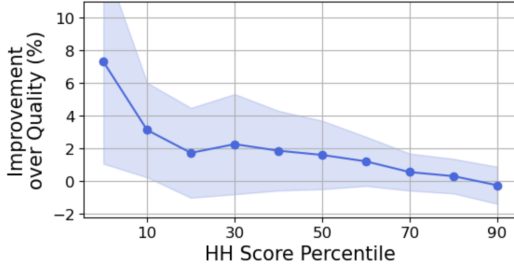


Figure 3: Improvement (averaged over the five datasets) in HH Score of QDIT over Quality-based selection.

ate the worst case performance with pairwise comparison by measuring the winning score on the hardest 10% of prompts according to HH score. Best case performance is measured in a similar manner. Measuring best and worst case performance provides more detailed insights into how diversity and quality affect model robustness.

◇ **Baselines.** We primarily compare the QDIT algorithm with two baselines: random data selection and quality based selection. For the Alpaca dataset, the quality baseline is trained on the same data as

Alpagasus (Chen et al., 2023).

4.3 Main Results

The main results can be found in Table 1 for large datasets and Table 2 for small datasets.

Effect of Quality. Similar to prior works, we find that selecting data based on quality significantly improves average performance, improving the average winning score versus Alpaca 52K by 6.37% and the average HH score by 11.6% when compared to random selection. However, we also find that selecting based on quality alone can hurt worst case performance, decreasing the lowest 10% winning score in two out of five settings compared to random data selection. We hypothesize that this performance drop is due to the fact that quality based selection hurts dataset diversity.

Effect of Diversity. On the other hand, we find that data selection with QDIT is able to achieve both a high average HH score (QDIT improves upon quality driven selection by 4.17% for winning score vs Alpaca 52K and improves average HH score by 1.5%) while achieving a much bet-

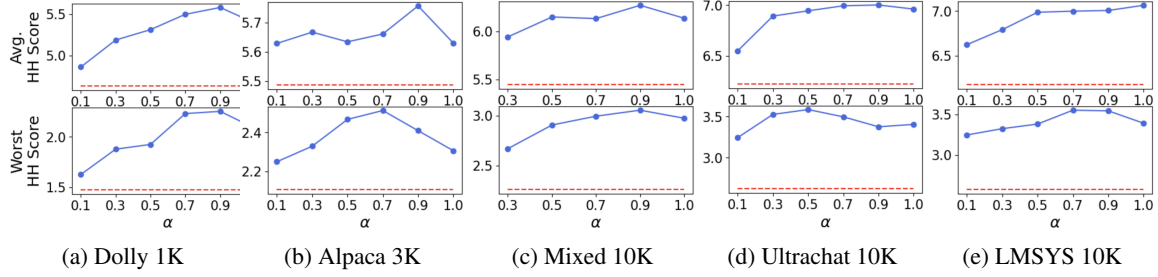


Figure 4: Effect of α on best case and worst case performance. The red line represents a randomly selected dataset and $\alpha = 1.0$ is quality-driven data selection. Worst HH Score refers to the bottom 10 percent of HH scores.

ter worst case performance than both random and quality-driven selection. In particular, we find that QDIT improves worst case HH score by 5.2% and worst winning score vs Alpaca 52K by 6.26% when compared to quality-driven data selection. This trend can be seen in Figure 3, where QDIT improves most over quality selection in the lowest and middle percentiles, while not affecting the highest percentiles. We hypothesize that QDIT’s more diverse dataset teaches the model to respond to a wide range of instructions, thereby decreasing the probability that it fails to follow evaluation instructions. Aggregated across different datasets the gain in robustness passes a paired t-test ($p < 0.05$). We provide more details and experiments on more random seeds in Appendix I.

From these experiments we conclude that by increasing data diversity while maintaining data quality, QDIT can improve instruction following capability compared to quality-driven selection. We note that improvements in worst-case performance are typically more important than improvements in best-case performance, as a high worst case performance will ensure users have a consistently positive experience.

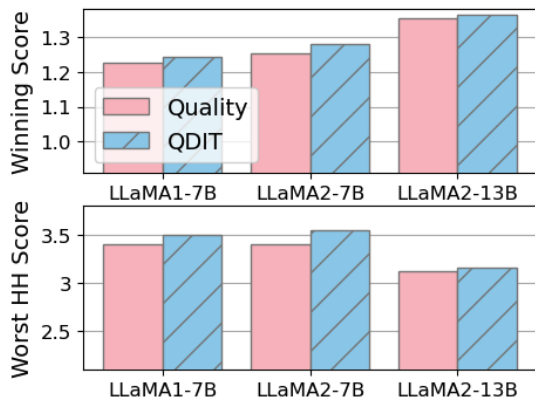


Figure 5: QDIT with different base models.

4.4 Analysis

Effect of α . We plot the effect of α on average and worst case performance in Figure 4. From Figure 4, we can see that the performance of QDIT changes smoothly with respect to α , indicating that QDIT is relatively robust to the value of α . In particular, values of $\alpha \in \{0.5, 0.7, 0.9\}$ typically having the highest worst case and average performance. However, decreasing α by too much ($\alpha = 0.1$) will result in a significant drop in performance, as will increasing α by too much ($\alpha = 1$). This highlights the need for a careful tradeoff between dataset quality and diversity.

Benchmark Performance. For a more comprehensive evaluation of QDIT, we follow Chia et al. (2023) and Gao et al. (2021) by evaluating our model on various benchmark datasets including MMLU (Hendrycks et al., 2020), BBH (Suzgun et al., 2022), DROP (Dua et al., 2019), ARC (Clark et al., 2018), LAMBADA (Paperno et al., 2016), and SCIQ (Welbl et al., 2017). The results can be found in Table 3. Details on our evaluation strategy can be found in Appendix G. Although these benchmarks are not fully aligned with instruction following ability, we find that QDIT typically improves benchmark performance compared to quality-driven selection, achieving a higher average score on four out of five datasets.

Different Base Models. We evaluate the performance of QDIT with different base models in Figure 5. From Figure 5 we find that QDIT can improve both average performance and worst case performance for other base models, including the more powerful LLaMA-2-13B. We remark that LLaMA-2-13B is only trained for 2 epochs, possibly resulting in a lower HH score.

Data Size. We evaluate the performance of QDIT with different training data sizes in Figure 7, where we find that QDIT leads to the highest gains in low-data regimes, but can still improve performance for

Table 3: Evaluation on benchmark datasets. We bold the best result out of quality based selection and QDIT. The α values are those used in Tables 1 and 2.

	Ultrachat 10K							LMSYS 10K						
	MMLU	BBH	ARC	DROP	LAMBADA	SCIQ	AVG	MMLU	BBH	ARC	DROP	LAMBADA	SCIQ	AVG
Random	32.12	33.19	58.34	26.24	69.77	85.4	50.84	33.05	32.57	60.15	25.06	68.5	86.7	51.01
Quality	35.44	32.06	60.34	17.01	70.38	85.8	50.17	34.74	32.32	58.54	25.95	68.58	82.6	50.46
QDIT	36.13	32.12	60.71	26.73	69.8	86.8	52.05	37.34	32.52	61.44	26.41	69.28	85.0	52.0

	Alpaca 3K							Mixed 10K						
	MMLU	BBH	ARC	DROP	LAMBADA	SCIQ	AVG	MMLU	BBH	ARC	DROP	LAMBADA	SCIQ	AVG
Random	36.17	30.25	61.67	26.32	71.64	87.0	52.18	32.93	30.92	58.34	20.33	68.1	84.1	49.12
Quality	34.71	29.97	60.99	19.62	69.85	82.7	49.64	33.07	31.38	60.34	26.37	69.4	88.4	51.49
QDIT	35.47	30.44	61.95	27.02	69.68	84.1	51.44	34.29	31.23	60.71	26.00	69.72	89.8	51.96

	Dolly 1K						
	MMLU	BBH	ARC	DROP	LAMBADA	SCIQ	AVG
Random	28.11	27.27	59.39	17.26	71.74	80.7	47.41
Quality	33.61	29.95	60.43	24.69	72.22	82.8	50.62
QDIT	33.78	30.33	59.84	22.59	72.26	80.6	49.9

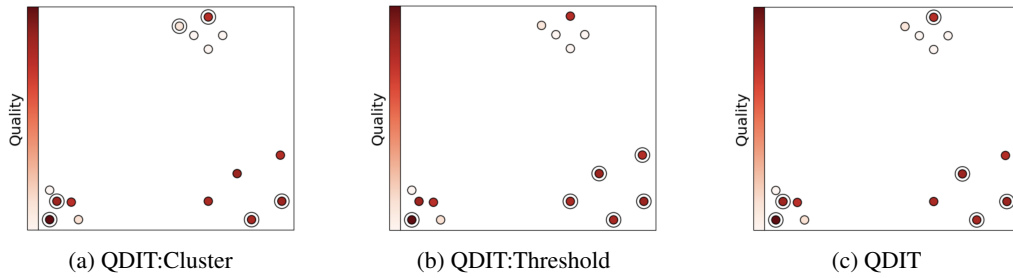


Figure 6: The selection strategy of the various QDIT algorithms on an example dataset. Six data points are selected and the selected data points are circled.

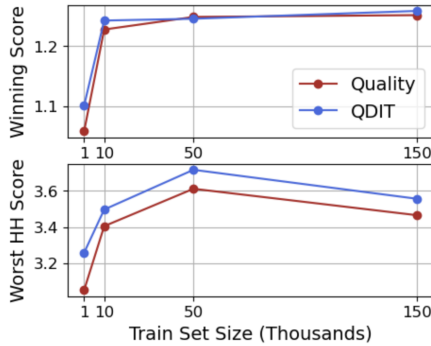


Figure 7: Ablation with different data sizes. The dataset is Ultrachat.

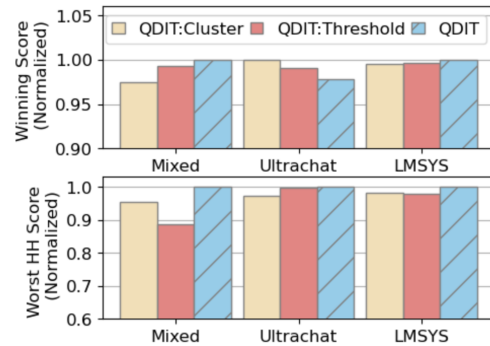


Figure 8: Performance of different QDIT variants on Ultrachat. Winning Score is versus Alpaca 52K.

larger datasets. We also find that increasing the dataset size beyond 50K only results in marginal gains of average performance, and can even decrease worst case HH score. This is likely due to the fact that the average reward score of the dataset decreases as the dataset size increases.

QDIT Variants. We study two variants of the QDIT algorithm that also attempt to improve data quality and diversity. The first variant, QDIT:Threshold, first orders the data point by quality, and then iteratively selects instructions that do not have a similarity score greater than τ with any

point included selected subset. This algorithm essentially de-duplicates the dataset, and is similar to algorithms used in Wang et al. (2022a) and Liu et al. (2023). The second variant we propose is QDIT:Cluster, in which we first cluster the instructions and then select an equal amount of data points from each cluster in a quality-driven manner. Comprehensive details can be found in Appendix H.

We demonstrate how each variant selects algorithms in Figure 6. From these figures, we can see that the clustering-based approach may not work well as some clusters may be low quality, resulting

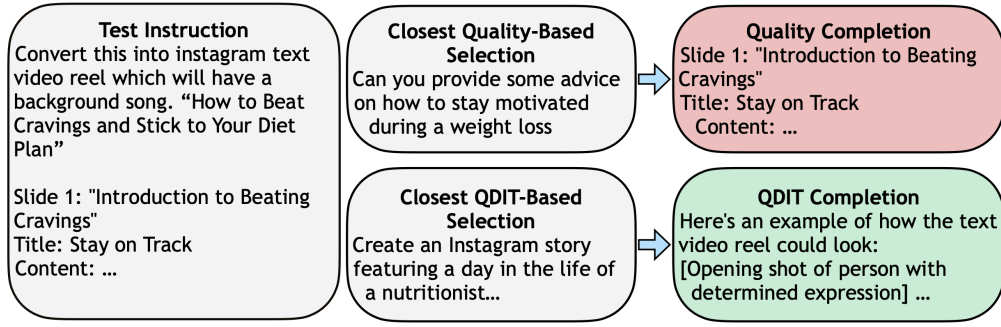


Figure 9: Case study on instruction generalization.

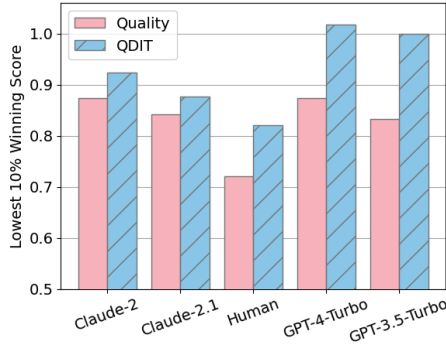


Figure 10: Performance of QDIT with different evaluators. The dataset is Alpaca.

in low-quality points being chosen. Moreover, the success of this variant is highly dependent on the success of clustering, which is difficult on large and imbalanced instruction tuning datasets. On the other hand, QDIT:Threshold will succeed in de-duplicating the dataset, but may fail to select a sufficiently diverse subset.

Experimentally, we observe in Figure 8 that the two variants of QDIT achieve slightly worse average performance compared to QDIT, but they often result in much worse worst-case performance.

Different Evaluators. To further investigate our finding that data diversity can improve instruction following performance, we compute the worst-case winning score with different evaluators. Concretely, we use Claude-2, Claude-2.1, GPT-3.5-Turbo, and GPT-4-Turbo as different model based evaluators (Achiam et al., 2023). We also conduct a blind human preference study, using the authors as annotators (see Appendix F). The results using different models as evaluators can be seen in Figure 10. From these results we can see QDIT consistently improves worst-case performance.

Reducing the Train-Test Gap. One explanation for QDIT’s improvement in instruction following capabilities is a reduction in the gap between the

training and testing data. More concretely, a model trained on a diverse dataset will be exposed to training instructions similar to those in the test set, and will therefore perform better on the test set.

A qualitative example of this phenomena can be found in Figure 9. In this example, the instruction asks for help creating a short video. The closest example (measured by cosine similarity of the instruction embedding) in the quality-based dataset is unrelated to this task, while the closest instruction in the QDIT-based dataset is a similar instruction asking how to create a short video on nutrition. As an end result, the QDIT model provides useful video suggestions, while the Quality-driven model merely repeats the provided video requirements.

In all of our experiments we observe that QDIT selects datasets that better cover the testing dataset compared to quality-based selection. This can be seen in Figure 11, where we observe that the QDIT selects more similar instruction to the test instructions than quality-based selection does. This phenomenon provides one explanation on how QDIT improves instruction following ability, but in general it is difficult to directly attribute instruction following capabilities to test-set coverage.

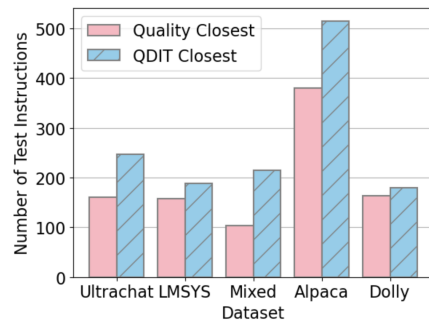


Figure 11: Number of test instructions where quality based selection or QDIT contains the closest training example. Ties are not included.

5 Limitations and Risks

This paper seeks to understand how dataset diversity and quality impact instruction tuning performance. We consider several datasets, base models, and benchmarks. Some limitations of our study are that we do not consider extremely large scale models (e.g. 70B), we assume a good measure of instruction quality (i.e. reward model) is available, and we only consider supervised finetuning, which is only one part of LLM alignment. To further improve upon the papers limitations, we would like to try scaling our experiments to even larger model sizes (e.g. 70B). In addition, it would be interesting to conduct our study on more sophisticated alignment techniques such as RLHF.

Our work is largely foundational and we therefore do not see any direct risks stemming from our work. However, it is possible that our method could be adapted to select harmful data to train on.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*.
- Gantavya Bhatt, Yifang Chen, Arnav M Das, Jifan Zhang, Sang T Truong, Stephen Mussmann, Yinglun Zhu, Jeffrey Bilmes, Simon S Du, Kevin Jamieson, et al. 2024. An experimental design framework for label-efficient supervised finetuning of large language models. *arXiv preprint arXiv:2401.06692*.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Lichang Chen, Shiyang Li, Jun Yan, Hai Wang, Kalpa Gunaratna, Vikas Yadav, Zheng Tang, Vijay Srivasan, Tianyi Zhou, Heng Huang, et al. 2023. Alpaga: Training a better alpaca with fewer data. *arXiv preprint arXiv:2307.08701*.
- Yew Ken Chia, Pengfei Hong, Lidong Bing, and Soujanya Poria. 2023. Instructeval: Towards holistic evaluation of instruction-tuned large language models. *arXiv preprint arXiv:2306.04757*.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.
- Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free dolly: Introducing the world’s first truly open instruction-tuned llm.
- Gérard Cornuéjols, George Nemhauser, and Laurence Wolsey. 1983. The uncappeditated facility location problem. Technical report, Cornell University Operations Research and Industrial Engineering.
- Arnav Das, Gantavya Bhatt, Megh Bhalerao, Vianne Gao, Rui Yang, and Jeff Bilmes. 2023. Accelerating batch active learning using continual learning techniques. *arXiv preprint arXiv:2305.06408*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*.
- Hanze Dong, Wei Xiong, Deepanshu Goyal, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. 2023a. Raft: Reward ranked finetuning for generative foundation model alignment. *arXiv preprint arXiv:2304.06767*.
- Yi Dong, Zhilin Wang, Makesh Narsimhan Sreedhar, Xianchao Wu, and Oleksii Kuchaiev. 2023b. Steerlm: Attribute conditioned sft as an (user-steerable) alternative to rlhf. *arXiv preprint arXiv:2310.05344*.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*.
- Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Alpaca-farm: A simulation framework for methods that learn from human feedback. *Preprint*, arXiv:2305.14387.
- Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff,

606	et al. 2021. A framework for few-shot language	Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Luke	660
607	model evaluation. <i>Version v0. 0.1. Sept.</i>	Zettlemoyer, Omer Levy, Jason Weston, and Mike	661
608	Xinyang Geng, Arnav Gudibande, Hao Liu, Eric Wal-	Lewis. 2023b. Self-alignment with instruction back-	662
609	lace, Pieter Abbeel, Sergey Levine, and Dawn Song.	translation. <i>arXiv preprint arXiv:2308.06259.</i>	663
610	2023. Koala: A dialogue model for academic re-	Wei Liu, Weihao Zeng, Keqing He, Yong Jiang, and	664
611	search . Blog post.	Junxian He. 2023. What makes good data for	665
612	Arnav Gudibande, Eric Wallace, Charlie Snell, Xinyang	alignment? a comprehensive study of automatic	666
613	Geng, Hao Liu, Pieter Abbeel, Sergey Levine, and	data selection in instruction tuning. <i>arXiv preprint</i>	667
614	Dawn Song. 2023. The false promise of imitating	<i>arXiv:2312.15685.</i>	668
615	proprietary llms. <i>arXiv preprint arXiv:2305.15717.</i>	Adyasha Maharana, Prateek Yadav, and Mohit Bansal.	669
616	Pengcheng He, Xiaodong Liu, Jianfeng Gao, and	2023. D2 pruning: Message passing for balancing di-	670
617	Weizhu Chen. 2020. Deberta: Decoding-enhanced	versity and difficulty in data pruning. <i>arXiv preprint</i>	671
618	bert with disentangled attention. <i>arXiv preprint</i>	<i>arXiv:2310.07931.</i>	672
619	<i>arXiv:2006.03654.</i>	Michel Minoux. 2005. Accelerated greedy algorithms	673
620	Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou,	for maximizing submodular set functions. In <i>Opti-</i>	674
621	Mantas Mazeika, Dawn Song, and Jacob Steinhardt.	mization Techniques: Proceedings of the 8th IFIP	675
622	2020. Measuring massive multitask language under-	Conference on Optimization Techniques Würzburg,	676
623	standing. <i>arXiv preprint arXiv:2009.03300.</i>	September 5–9, 1977, pages 234–243. Springer.	677
624	Hamish Ivison, Yizhong Wang, Valentina Pyatkin,	Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru,	678
625	Nathan Lambert, Matthew Peters, Pradeep Dasigi,	Amin Karbasi, Jan Vondrák, and Andreas Krause.	679
626	Joel Jang, David Wadden, Noah A Smith, Iz Belt-	2015. Lazier than lazy greedy. In <i>Proceedings of</i>	680
627	agy, et al. 2023. Camels in a changing climate: En-	<i>the AAAI Conference on Artificial Intelligence</i> , vol-	681
628	hancing lm adaptation with tulu 2. <i>arXiv preprint</i>	ume 29.	682
629	<i>arXiv:2311.10702.</i>	Swaroop Mishra, Daniel Khashabi, Chitta Baral, and	683
630	Katrin Kirchhoff and Jeff Bilmes. 2014. Submodularity	Hannaneh Hajishirzi. 2021. Cross-task generaliza-	684
631	for data selection in machine translation. In <i>Proceed-</i>	tion via natural language crowdsourcing instructions.	685
632	<i>ings of the 2014 Conference on Empirical Methods</i>	<i>arXiv preprint arXiv:2104.08773.</i>	686
633	<i>in Natural Language Processing (EMNLP)</i> , pages	George L Nemhauser, Laurence A Wolsey, and Mar-	687
634	131–141.	shall L Fisher. 1978. An analysis of approximations	688
635	Nikita Kitaev, Steven Cao, and Dan Klein. 2019. Multi-	for maximizing submodular set functions—i. <i>Mathe-</i>	689
636	lingual constituency parsing with self-attention and	<i>matical programming</i> , 14:265–294.	690
637	pre-training . In <i>Proceedings of the 57th Annual Meet-</i>	Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida,	691
638	<i>ing of the Association for Computational Linguistics</i> ,	Carroll Wainwright, Pamela Mishkin, Chong Zhang,	692
639	pages 3499–3505, Florence, Italy. Association for	Sandhini Agarwal, Katarina Slama, Alex Ray, et al.	693
640	Computational Linguistics.	2022. Training language models to follow instruc-	694
641	Nikita Kitaev and Dan Klein. 2018. Constituency pars-	tions with human feedback. <i>Advances in Neural</i>	695
642	ing with a self-attentive encoder . In <i>Proceedings</i>	<i>Information Processing Systems</i> , 35:27730–27744.	696
643	<i>of the 56th Annual Meeting of the Association for</i>	Denis Paperno, Germán Kruszewski, Angeliki Lazari-	697
644	<i>Computational Linguistics (Volume 1: Long Papers)</i> ,	dou, Quan Ngoc Pham, Raffaella Bernardi, Sandro	698
645	pages 2676–2686, Melbourne, Australia. Association	Pezzelle, Marco Baroni, Gemma Boleda, and Raquel	699
646	for Computational Linguistics.	Fernández. 2016. The lambada dataset: Word pre-	700
647	Ashutosh Kumar, Satwik Bhattamishra, Manik Bhan-	diction requiring a broad discourse context. <i>arXiv</i>	701
648	dari, and Partha Talukdar. 2019. Submodular	<i>preprint arXiv:1606.06031.</i>	702
649	optimization-based diverse paraphrasing and its ef-	Baolin Peng, Chunyuan Li, Pengcheng He, Michel Gal-	703
650	fectiveness in data augmentation. In <i>Proceedings of</i>	ley, and Jianfeng Gao. 2023. Instruction tuning with	704
651	<i>the 2019 Conference of the North American Chap-</i>	gpt-4. <i>arXiv preprint arXiv:2304.03277.</i>	705
652	<i>ter of the Association for Computational Linguistics:</i>	Alec Radford, Jeffrey Wu, Rewon Child, David Luan,	706
653	<i>Human Language Technologies, Volume 1 (Long and</i>	Dario Amodei, Ilya Sutskever, et al. 2019. Language	707
654	<i>Short Papers)</i> , pages 3609–3619.	models are unsupervised multitask learners. <i>OpenAI</i>	708
655	Ming Li, Yong Zhang, Zhitao Li, Jiuhai Chen, Lichang	<i>blog</i> , 1(8):9.	709
656	Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and	Nils Reimers and Iryna Gurevych. 2019. Sentence-bert:	710
657	Jing Xiao. 2023a. From quantity to quality: Boosting	Sentence embeddings using siamese bert-networks.	711
658	llm performance with self-guided data selection for	In <i>Proceedings of the 2019 Conference on Empirical</i>	712
659	instruction tuning. <i>arXiv preprint arXiv:2308.12032.</i>	<i>Methods in Natural Language Processing</i> . Associa-	713
		tion for Computational Linguistics.	714

715	Victor Sanh, Albert Webson, Colin Raffel, Stephen H	Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin	772
716	Bach, Lintang Sutawika, Zaid Alyafeai, Antoine	Guu, Adams Wei Yu, Brian Lester, Nan Du, An-	773
717	Chaffin, Arnaud Stiegler, Teven Le Scao, Arun	drew M Dai, and Quoc V Le. 2021. Finetuned lan-	774
718	Raja, et al. 2021. Multitask prompted training en-	guage models are zero-shot learners. <i>arXiv preprint</i>	775
719	ables zero-shot task generalization. <i>arXiv preprint</i>	<i>arXiv:2109.01652</i> .	776
720	<i>arXiv:2110.08207</i> .		
721	Mirac Suzgun, Nathan Scales, Nathanael Schärli, Se-	Johannes Welbl, Nelson F Liu, and Matt Gardner. 2017.	777
722	bastian Gehrmann, Yi Tay, Hyung Won Chung,	Crowdsourcing multiple choice science questions.	778
723	Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny	<i>arXiv preprint arXiv:1707.06209</i> .	779
724	Zhou, et al. 2022. Challenging big-bench tasks and		
725	whether chain-of-thought can solve them. <i>arXiv</i>	Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng,	780
726	<i>preprint arXiv:2210.09261</i> .	Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin	781
		Jiang. 2023. Wizardlm: Empowering large lan-	782
		guage models to follow complex instructions. <i>arXiv</i>	783
727	Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann	<i>preprint arXiv:2304.12244</i> .	784
728	Dubois, Xuechen Li, Carlos Guestrin, Percy Liang,		
729	and Tatsunori B. Hashimoto. 2023. Stanford alpaca:	Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle	785
730	An instruction-following llama model. https://	Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang,	786
731	github.com/tatsu-lab/stanford_alpaca .	Zhuohan Li, Zi Lin, Eric. P Xing, Joseph E. Gonzal-	787
		ez, Ion Stoica, and Hao Zhang. 2023. <i>Lmsys-chat-</i>	788
732	Hugo Touvron, Louis Martin, Kevin Stone, Peter Al-	<i>lm: A large-scale real-world llm conversation dataset.</i>	789
733	bert, Amjad Almahairi, Yasmine Babaei, Nikolay	<i>Preprint</i> , arXiv:2309.11998.	790
734	Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti		
735	Bhosale, et al. 2023. Llama 2: Open founda-	Chunting Zhou, Pengfei Liu, Puxin Xu, Srinu Iyer, Jiao	791
736	tion and fine-tuned chat models. <i>arXiv preprint</i>	Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu,	792
737	<i>arXiv:2307.09288</i> .	Lili Yu, et al. 2023. Lima: Less is more for alignment.	793
		<i>arXiv preprint arXiv:2305.11206</i> .	794
738	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob		
739	Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz		
740	Kaiser, and Illia Polosukhin. 2017. Attention is all		
741	you need. <i>Advances in neural information processing</i>		
742	<i>systems</i> , 30.		
743	Peiqi Wang, Yikang Shen, Zhen Guo, Matthew Stal-		
744	lone, Yoon Kim, Polina Golland, and Rameswar		
745	Panda. 2024. Diversity measurement and subset se-		
746	lection for instruction tuning datasets. <i>arXiv preprint</i>		
747	<i>arXiv:2402.02318</i> .		
748	Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack		
749	Hessel, Tushar Khot, Khyathi Raghavi Chandu,		
750	David Wadden, Kelsey MacMillan, Noah A Smith,		
751	Iz Beltagy, et al. 2023a. How far can camels go?		
752	exploring the state of instruction tuning on open re-		
753	sources. <i>arXiv preprint arXiv:2306.04751</i> .		
754	Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Al-		
755	isa Liu, Noah A Smith, Daniel Khashabi, and Han-		
756	nanah Hajishirzi. 2022a. Self-instruct: Aligning lan-		
757	guage model with self generated instructions. <i>arXiv</i>		
758	<i>preprint arXiv:2212.10560</i> .		
759	Yizhong Wang, Swaroop Mishra, Pegah Alipoor-		
760	molabashi, Yeganeh Kordi, Amirreza Mirzaei,		
761	Anjana Arunkumar, Arjun Ashok, Arut Selvan		
762	Dhanasekaran, Atharva Naik, David Stap, et al.		
763	2022b. Super-naturalinstructions: Generalization via		
764	declarative instructions on 1600+ nlp tasks. <i>arXiv</i>		
765	<i>preprint arXiv:2204.07705</i> .		
766	Zhilin Wang, Yi Dong, Jiaqi Zeng, Virginia Adams,		
767	Makesh Narsimhan Sreedhar, Daniel Egert, Olivier		
768	Delalleau, Jane Polak Scowcroft, Neel Kant, Aidan		
769	Swope, et al. 2023b. Helpsteer: Multi-attribute		
770	helpfulness dataset for steerlm. <i>arXiv preprint</i>		
771	<i>arXiv:2311.09528</i> .		

A Similarity Metrics

In QDIT, we use the cosine similarity of instruction embeddings as the similarity function in (1), where the instruction embeddings are computed with sentence transformers (Reimers and Gurevych, 2019). More specifically, we use the all-mpnet-base-v2 model available from Huggingface.

B Computational Cost of QDIT

We measure the wall time of QDIT as taking approximately 9.42 minutes to select a dataset of size 10000. This experiment was conducted on a single A100 40G GPU. In contrast, training on Ultrachat 10K takes 48 minutes on 8 A100 40G GPUs, meaning that data selection (given the embeddings and reward score) has 2% of the cost of training. Taking into account the inference and evaluation process, our method has an even smaller relative cost. We remark that the reward score and embedding generation can typically be done on a small GPU very quickly, or it can even be done on a CPU for minimal cost.

C Tradeoff between Quality and Diversity: Additional Results

We display additional results on the effect of α on different datasets diversity and quality in Figures 12, 13, 14. We again find that there is a tradeoff between quality and diversity, and that α can be used to control this tradeoff.

D Training Details

We display the hyperparameter details in Table 4 and Table 5.

Table 4: General training details. The same hyperparameter setting is used for every dataset and data selection strategy, following Chen et al. (2023).

Batch Size	Learning Rate	Epochs	Max Length	Weight Decay
128	2×10^{-5}	3	512	0

Table 5: The hyperparameter α used in the main experiments.

Dolly	15K	Alpaca	52K	Mixed	270K	Ultrachat	1.3M	MLMSYS	1M
0.7	0.7	0.9	0.7	0.7	0.7	0.7	0.7	0.7	0.7

E Complete Win-Tie-Lose Results

In this appendix we show the win, ties, and losses achieved versus the reference models as judged by Claude 2. The results can be seen in Figures 15, 16, 17, 18, 19, and 20.

F Human Study Details

For the human study, we use a subset of the paper authors as annotators. For each example, we randomize the order of the reference model generation and evaluated model generation, to keep the study blind. We then ask the annotators to select their preferred generation according to the AlpacaFarm prompt (Dubois et al., 2023).

G Benchmark Environments

For evaluation on common NLP benchmarks, we follow Chia et al. (2023). In particular, we conduct five shot evaluation on MMLU, three shot on BBH, and three shot on DROP. For ARC, LAMBADA, and SCIQ, we use the default zero shot setting of (Gao et al., 2021).

H QDIT Variants

In this section we describe and analyze the variants of QDIT.

QDIT:Cluster. In this variant of QDIT, we first cluster all the instructions based on their sentence-transformer embeddings using the k -means algorithm, where $k = 100$. We then select an equal number of points from each cluster, and points are selected from each cluster based on quality.

QDIT:Threshold. In this variant of QDIT, we first sort the instructions based on quality. We then iteratively remove instructions from the selected dataset if they have a similarity with some other instruction in the dataset greater than a threshold τ . In our experiments we use the cosine similarity as the similarity metric and set $\tau = 0.5$.

I Sensitivity to Randomness

To further investigate QDIT’s sensitivity to random seeds, we conducted experiments over 3 seeds on the Alpaca dataset. Our findings (see Table 6) are consistent with the rest of our results: QDIT and Quality based selection significantly outperform random selection, and QDIT achieves much higher worst case performance than both Random 3K and Quality 3K. Also, we notice small standard devi-

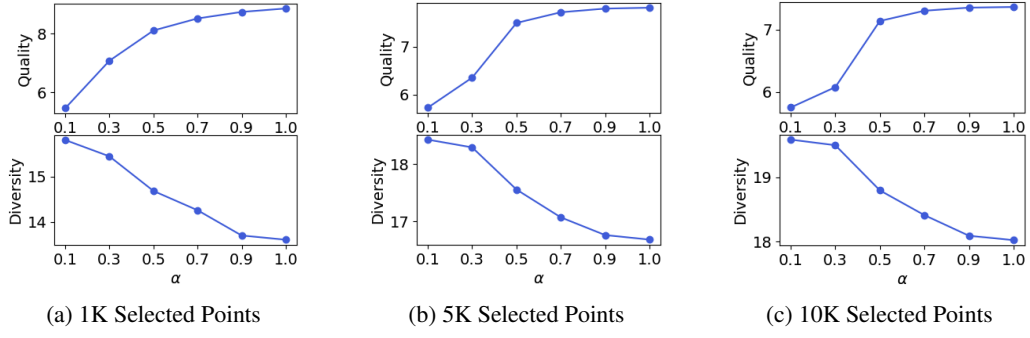


Figure 12: Effect of α on dataset quality and diversity. The dataset is Mixed 270K.

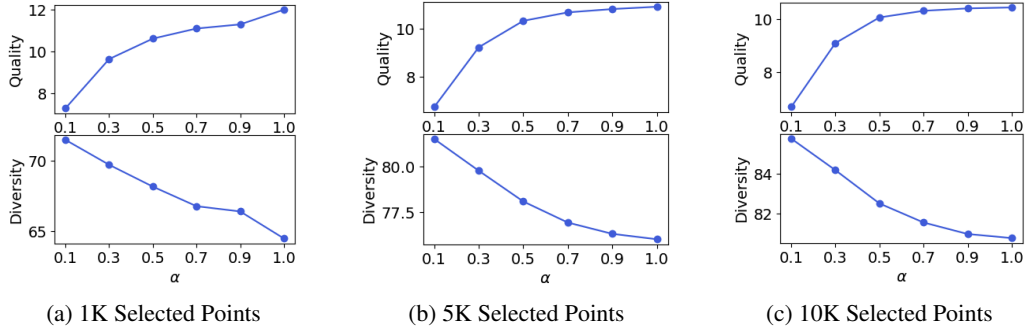


Figure 13: Effect of α on dataset quality and diversity. The dataset is Ultrachat 1.3M.

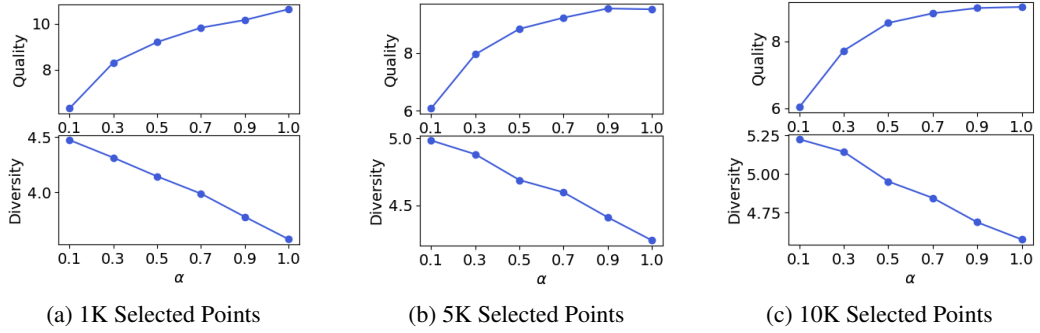


Figure 14: Effect of α on dataset quality and diversity. The dataset is LMSYS 1M.

ation, indicating our experimental framework is robust to the choice of random seed.

Table 6: Results with different random seeds.

Alpaca 3K	Avg. HH Score	Worst HH Score
Random 3K	5.55 (0.05)	2.12 (0.04)
Quality 3K	5.61 (0.01)	2.30 (0.02)
QDIT 3K	5.63 (0.02)	2.41 (0.07)

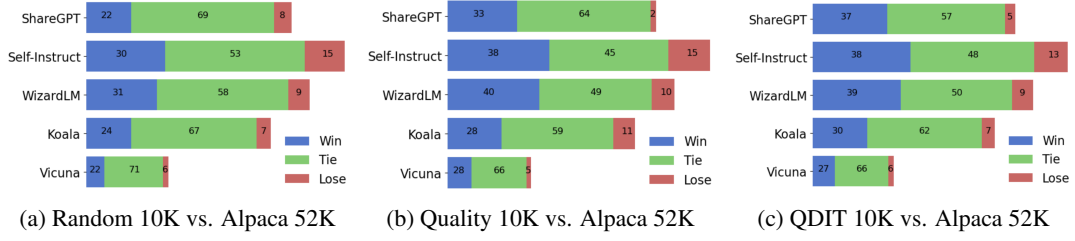


Figure 15: We display the results on Ultrachat as judged by Claude 2. The base model here is LLaMA-2 7B.

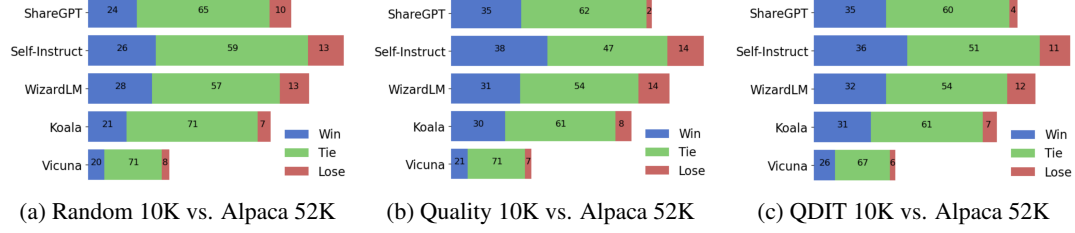


Figure 16: We display the results on Ultrachat as judged by Claude 2. The base model here is LLaMA-1 7B.

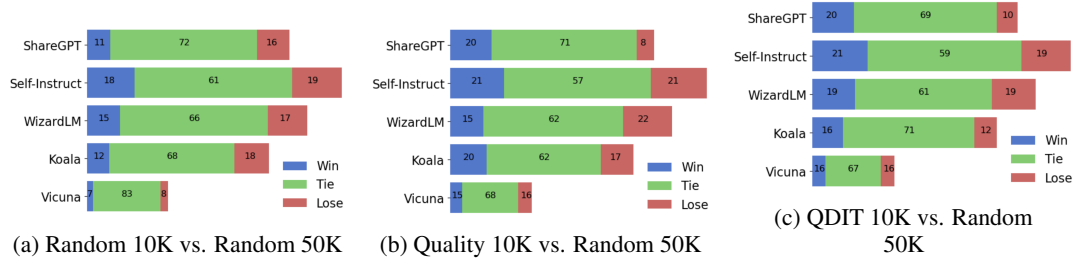


Figure 17: We display the results on Ultrachat as judged by Claude 2. The base model here is LLaMA-1 7B.

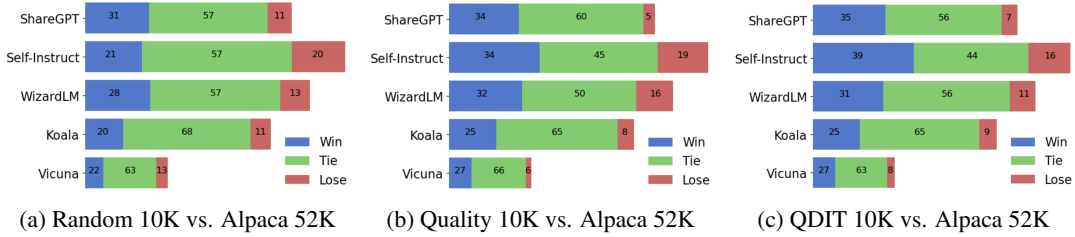


Figure 18: We display the results on LMSYS as judged by Claude 2. The base model here is LLaMA-1 7B.

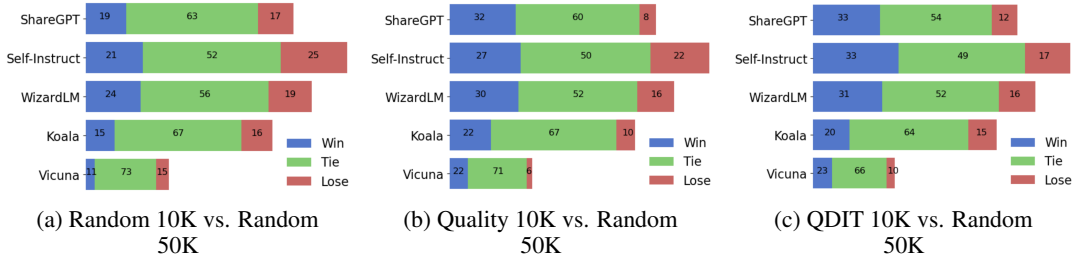


Figure 19: We display the results on LMSYS as judged by Claude 2. The base model here is LLaMA-1 7B.

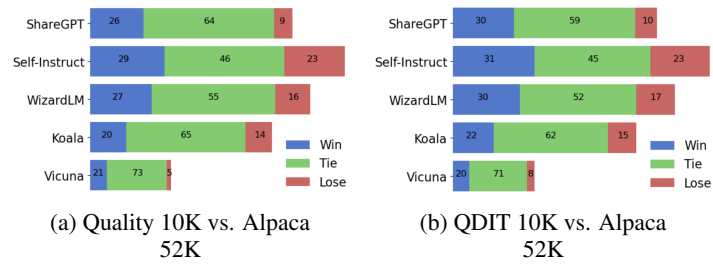


Figure 20: We display the results on Ultrachat as judged by Claude 2. The base model here is Mistral 7B.